

Pools (Pilas y Colas): Problema ABA

Pools (Estructuras de Datos)

¿Pool?

Un multiset

- **Permite duplicados** (un elemento puede aparecer más de una vez).
- **No** necesariamente incluye un método `contains()`.

La Interfaz `Pool<T>`

```
public interface Pool<T> {  
    void put(T item);  
    T get();  
}
```

Utilización típica

Como un buffer intermedio entre productores y consumidores.

Acotados (Bounded) vs. No Acotados (Unbounded):

- *Acotados*: Tienen **capacidad** limitada. Mantienen a los productores y consumidores sincronizados impidiendo que los productores se *alejen* mucho de los consumidores.
- *No acotados*: Capacidad ilimitada. Útiles si es difícil fijar un límite al ritmo del productor.

Métodos Parciales y Totales

Método Parcial

Las llamadas pueden esperar a que se cumplan ciertas condiciones.

- `get()` y `put()` se bloquean si no se pueden completar.
- **Uso:** Tiene sentido si el productor o consumidor tienen que esperar necesariamente.

Método Total

Las llamadas no esperan condiciones para ejecutar.

- Un `get()` `put()` retornan error o lanzan excepción si no se pueden completar.
- **Uso:** Cuando el proceso puede hacer otras cosas (probar con otro buffer).

Método Parcial Síncrono

Espera a que otro método se solape con su propio intervalo de ejecución.

- **Pool síncrono:**
 - El método que agrega un elemento se bloquea hasta que otra llamada lo remueve.
 - Simétricamente, el método que remueve un elemento se bloquea hasta que otra llamada lo hace disponible.
- Implementan *rendezvous* para intercambiar información.

Garantías de Equidad (Fairness) en Pools

Políticas de Acceso

Los pools ofrecen diferentes garantías de equidad para procesar elementos:

- **FIFO (First-In-First-Out):** Funciona como una cola. Los elementos se procesan en el orden estricto en que llegaron.
- **LIFO (Last-In-First-Out):** Funciona como una pila. El último elemento en entrar es el primero en salir.
- **Otras propiedades:** Políticas con garantías típicamente más débiles o aleatorias.

Colas Concurrentes (FIFO)

Definición de Queue $\langle T \rangle$

Una secuencia ordenada de elementos de tipo T que provee:

- **enq(x):** Agrega el elemento x al final de la cola (*tail*). Equivale a `put()`.
- **deq():** Remueve y devuelve el elemento al principio de la cola (*head*). Equivale a `get()`.

Correctitud

- **Linealizabilidad:** Una cola concurrente debe ser linealizable con respecto a una cola secuencial.

Nivel de Concurrency en Colas Acotadas

Interferencia entre Operaciones

¿Cuánta concurrencia podemos esperar con múltiples hilos encolando y desencolando?

- **enq() vs. deq():** Operan en extremos opuestos (*tail* y *head*). Si la cola no está vacía ni llena, deberían proceder sin interferirse.
- **Llamadas concurrentes del mismo tipo:** Múltiples enq() probablemente interfieran entre sí. Lo mismo ocurre entre múltiples deq().

Nivel de Concurrency en Colas Acotadas

Interferencia entre Operaciones

¿Cuánta concurrencia podemos esperar con múltiples hilos encolando y desencolando?

- **enq() vs. deq():** Operan en extremos opuestos (*tail* y *head*). Si la cola no está vacía ni llena, deberían proceder sin interferirse.
- **Llamadas concurrentes del mismo tipo:** Múltiples enq() probablemente interfieran entre sí. Lo mismo ocurre entre múltiples deq().

El Desafío de la Implementación

Aunque este razonamiento informal suena convincente y es mayormente correcto, **lograr este nivel de concurrencia en la práctica no es trivial.**

Representación Secuencial de la Cola Acotada

```
public class SequentialBoundedQueue<T> {  
    volatile Node head, tail;  
    int size;  
    final int capacity;  
  
    public SequentialBoundedQueue(int c) {  
        capacity = c;  
        head = new Node(null);  
        tail = head;  
        size = 0;  
    }  
}
```

```
protected class Node {  
    public T value;  
    public volatile Node next;  
    public Node(T x) {  
        value = x;  
        next = null;  
    }  
}
```

Invariante de representación

- head puede contener null. Los restantes nodos no.
- tail es alcanzable y desde head.
- capacity es la cantidad de nodos alcanzables desde head.

Estructura de la Cola Concurrente Acotada

```
public class BoundedQueue<T> {
    ReentrantLock enqLock, deqLock;
    Condition notEmptyCondition;
    Condition notFullCondition;
    AtomicInteger size;
    volatile Node head, tail;
    final int capacity;

    public BoundedQueue(int c) {
        capacity = c;
        head = new Node(null);
        tail = head;
        size = new AtomicInteger(0);
        enqLock = new ReentrantLock();
        notFullCondition = enqLock.newCondition();
        deqLock = new ReentrantLock();
        notEmptyCondition = deqLock.newCondition();
    }
}
```

```
protected class Node {
    public T value;
    public volatile Node next;

    public Node(T x) {
        value = x;
        next = null;
    }
}
```

Representación

- **Dos Locks** `enqLock` y `deqLock`
 - Permite que un productor y un consumidor operen simultáneamente.
 - Evita que los procesos que encolan bloqueen innecesariamente a los que desencolan y viceversa.
- **Variables de Condición**
 - `notFullCondition`: Asociada a `enqLock`, para notificar que hay espacio.
 - `notEmptyCondition`: Asociada a `deqLock`, para notificar que hay elementos.
- **Control de Capacidad**
 - `size` es un `AtomicInteger` que mantiene la cantidad de elementos en la cola.
 - Es **atómico** porque va a ser accedido sin protección.

Implementación del Método enq(x)

```
public void enq(T x) {  
    boolean mustWakeDequeuers = false;  
    Node e = new Node(x);  
    enqLock.lock();  
    try {  
        while (size.get() == capacity)  
            notFullCondition.await();  
        tail.next = e;  
        tail = e;  
        if (size.getAndIncrement() == 0)  
            mustWakeDequeuers = true;  
    } finally {  
        enqLock.unlock();  
    }  
    if (mustWakeDequeuers) {  
        deqLock.lock();  
        try {  
            notEmptyCondition.signalAll();  
        } finally {  
            deqLock.unlock();  
        }  
    }  
}
```

- **Lock Dividido:** enqLock protege el final de la cola.
- **Espera:** Si la cola está llena, el hilo se bloquea en notFullCondition.
- **Wake-ups:** Sólo despierta a los consumidores (deqLock) si la cola pasa de estar vacía (size == 0) a tener un elemento.

Implementación del Método `deq()`

```
public T deq() {
    T result;
    boolean mustWakeEnqueuers = false;
    deqLock.lock();
    try {
        while (head.next == null)
            notEmptyCondition.await();
        result = head.next.value;
        head = head.next;
        if (size.getAndDecrement() == capacity) {
            mustWakeEnqueuers = true;
        }
    } finally { deqLock.unlock(); }
    if (mustWakeEnqueuers) {
        enqLock.lock();
        try {
            notFullCondition.signalAll();
        } finally { enqLock.unlock(); }
    }
    return result;
}
```

- **Uso del Centinela:** El valor se extrae de `head.next.value`. Luego, ese nodo se convierte en la nueva cabeza (`head`), descartando el valor viejo.
- **Espera:** Si `head.next` es nulo, la cola está vacía y el hilo espera en `notEmptyCondition`.
- **Optimización de Señal:** Solo despierta a los productores (`enqLock`) si la cola pasa de estar completamente llena (`size == capacity`) a liberar un lugar.

Por qué no hay Lost-Wakeup

Progreso del Productor

- Después de determinar que hay espacio, **encolar** termina con éxito.
- Ningún otro proceso puede llenar la cola porque los demás productores están bloqueados y los consumidores solo incrementan el espacio libre.

Por qué no hay Lost-Wakeup

Progreso del Productor

- Después de determinar que hay espacio, **encolar** termina con éxito.
- Ningún otro proceso puede llenar la cola porque los demás productores están bloqueados y los consumidores solo incrementan el espacio libre.

Sincronización

- El productor detecta la cola llena en dos pasos: evalúa `size` y luego espera en la condición).
- Aunque `size` no usa locks, el proceso que desencola debe adquirir `enqLock` antes de hacer `signal`.
- **Garantía:** Esto impide que el `signal` pueda ocurrir en el medio de los dos pasos del productor.

Método `deq()`

Detección de Cola Vacía

- **Verificación:** Comprueba si `head.next == null`.
 - El consumidor no lee el campo `size`.
- Si la cola está vacía, espera en `notEmptyCondition`.
- Al despertar, vuelve a verificar la condición antes de avanzar.

Extracción y Notificación

- **Exclusión Mutua:** La cola permanecerá no vacía si se confirma que no está vacía porque los demás consumidores están bloqueados.
- Se lee el valor del primer nodo `head.next` y se avanza el centinela.
- Decrementa `size`. Si la cola estaba llena, adquiere `enqLock` y despierta a los productores.

Punto de Linealización e Invariantes Temporales

Linealización de `enq()`

El elemento se agrega cuando se actualiza el puntero `tail.next`, no `tail`.

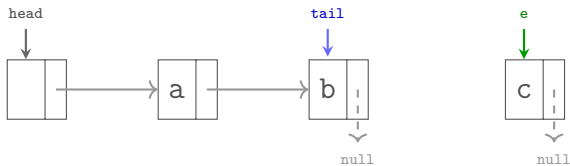
- **Linearización:** en la redirección de `tail.next = e`.
- **Desfase:** El elemento es parte de la cola aunque el proceso no haya actualizado `tail`.

Efectos

- Un consumidor puede avanzar antes de que el productor termine.
 - Desencolar puede extraer el nuevo nodo y avanzar `head` de inmediato.
- `size` puede ser **negativo** temporalmente si el consumidor decreuenta antes del incremento del productor.
- El productor no necesita despertar porque su elemento ya fue removido.

Inserción en la Cola (enq)

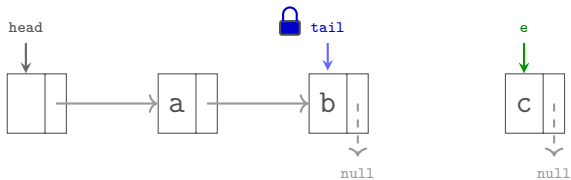
size = 2



- Se instancia en memoria el nuevo nodo **e** con el valor **b**.
- El puntero **tail** referencia al último elemento de la lista (**a**). Su enlace **next** es **null**.

Inserción en la Cola (enq)

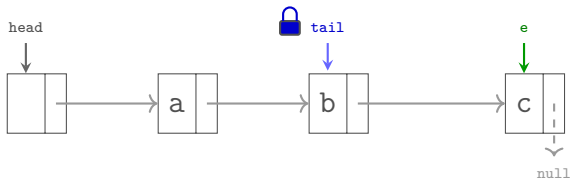
size = 2



- El hilo adquiere de manera exclusiva el `enqLock` para garantizar exclusión mutua.

Inserción en la Cola (enq)

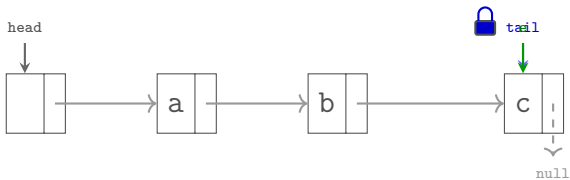
size = 2



- **Punto de Linealización:** Se ejecuta `tail.next = e`. El elemento pasa a pertenecer abstractamente a la cola.

Inserción en la Cola (enq)

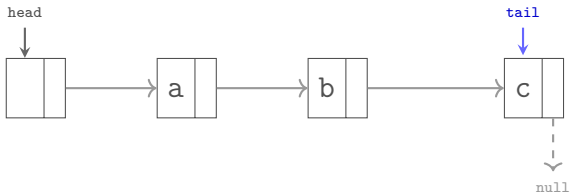
size = 3



- Se actualiza la referencia global `tail = e` y se incrementa el contador de estado atómico `size` a 2.

Remoción en la Cola (deq)

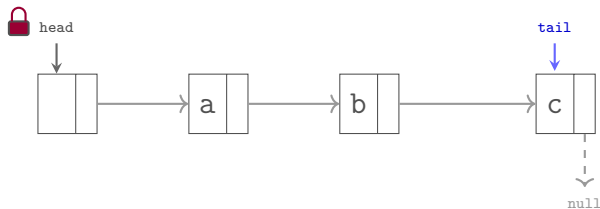
size = 3 | result = —



- Un hilo invoca a `deq()`. Se verifica que la cola no esté vacía comprobando que `head.next != null` (el nodo **a** existe).
- El nodo `head` actual opera puramente como centinela.

Remoción en la Cola (deq)

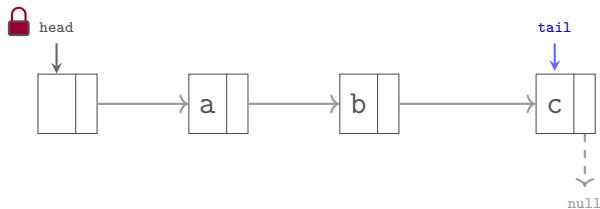
size = 3 | result = —



- El hilo consumidor adquiere de manera exclusiva el `deqLock` para proteger la estructura de la cabeza de la cola.

Remoción en la Cola (deq)

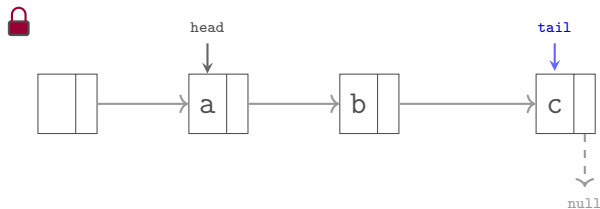
size = 3 | result = a



- Se lee el valor del primer nodo real de la cola: `result = head.next.value` (retorna el valor **a**).

Remoción en la Cola (deq)

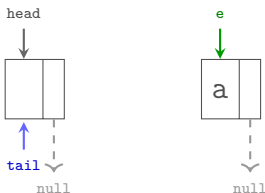
size = 2 | result = a



- **Punto de Linealización:** Se ejecuta `head = head.next`. El nodo que contenía **a** pasa a ser el nuevo centinela vacío.
- Se decrementa el contador `size` a 2 y se libera el bloqueo.

Anomalía Concurrente: size temporalmente en -1

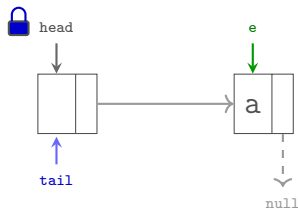
size = 0 | Operación: Cola Vacía (Inicial)



- **Estado inicial:** La cola está completamente vacía. El puntero `tail` apunta al centinela vacío y `size` es 0.

Anomalía Concurrente: size temporalmente en -1

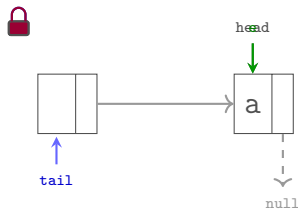
size = 0 | Operación: `enq(a)` enlazado estructuralmente



- **Productor interviene:** Se ejecuta `tail.next = e`. El elemento **a** queda enlazado físicamente (punto de linealización de `enq`), pero el hilo se suspende antes de avanzar `tail` o sumar a `size`.

Anomalía Concurrente: size temporalmente en -1

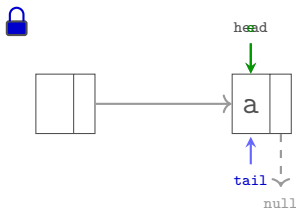
size = -1 | Operación: **deq()** **intercala** y decrementa



- **Consumidor se intercala:** Llama a `deq()`. En vez de leer el contador, comprueba si `head.next != null`. Como es verdadero (apunta a **a**), extrae el elemento, avanza `head` y decrementa: **size cae temporalmente a -1** .

Anomalía Concurrente: size temporalmente en -1

size = 0 | Operación: **enq(a)** despierta y finaliza



- **Productor finaliza:** El hilo original se despierta, mueve `tail = e` y ejecuta `size.getAndIncrement()`. El valor de `size` sube a 0 y la estructura vuelve a quedar consistente y vacía.

Cola Total No Acotada (UnboundedQueue)

Operaciones totales

- `enq(x)`: Siempre tiene éxito porque no tiene una capacidad acotada.
- `deq()` Es total. si la cola está vacía, no espera y lanza una excepción `EmptyException`.

Estructura de la Cola No Acotada

```
public class UnboundedQueue<T> {  
    ReentrantLock enqLock;  
    ReentrantLock deqLock;  
    volatile Node head, tail;  
  
    protected class Node {  
        public T value;  
        public volatile Node next;  
        public Node(T x) {  
            value = x;  
            next = null;  
        }  
    }  
}
```

```
public UnboundedQueue() {  
    head = new Node(null);  
    tail = head;  
    enqLock = new ReentrantLock();  
    deqLock = new ReentrantLock();  
}
```

Métodos Totales: enq() y deq()

```
public void enq(T x) {  
    Node e = new Node(x);  
    enqLock.lock();  
    try {  
        tail.next = e;  
        tail = e;  
    } finally {  
        enqLock.unlock();  
    }  
}
```

```
public T deq() throws  
    EmptyException {  
    T result;  
    deqLock.lock();  
    try {  
        if (head.next == null) {  
            throw new EmptyException();  
        }  
        result = head.next.value;  
        head = head.next;  
    } finally {  
        deqLock.unlock();  
    }  
    return result;  
}
```

Cola Total No Acotada: Análisis

Correctitud. Linealización

- `enq`: el elemento se añade cuando se redirige el puntero `next` de `tail`.
- `deq`: existoso: cuando se cambia `head`.
- `deq`: no existoso: cuando se chequea `head.next == null`.

Progreso

- Cada método adquiere únicamente un lock (`enqLock` o `deqLock`). Luego no puede haber **deadlocks**
- Si los locks son *fair*, libre de inanición.

Cola Libre de Bloqueos (LockFreeQueue): Representación

```
public class LockFreeQueue<T> {  
    AtomicReference<Node> head, tail;  
  
    public LockFreeQueue() {  
        Node node = new Node(null);  
        head = new AtomicReference(node);  
        tail = new AtomicReference(node);  
    }  
}
```

```
public class Node {  
    public T value;  
    public AtomicReference<Node> next;  
  
    public Node(T value) {  
        this.value = value;  
        next = new AtomicReference<Node>(null);  
    }  
}
```

Método enq()

```
public void enq(T value) {  
    Node node = new Node(value);  
    while (true) {  
        Node last = tail.get();  
        Node next = last.next.get();  
        if (last == tail.get()) {  
            if (next == null) {  
                if (last.next.CAS(next, node)) {  
                    tail.CAS(last, node);  
                    return;  
                }  
            } else {  
                tail.CAS(last, next);  
            }  
        }  
    }  
}
```

- Verificar que el último nodo no tenga sucesor (`next == null`).
- Intenta enlazar al nuevo nodo mediante un CAS. Si tiene éxito, la inserción es efectiva.
- Intenta adelantar `tail`. Si falla, significa que otra operación lo ayudó.
- **Ayuda:** Si se detecta que el nodo final tiene un sucesor, intenta adelantar el puntero `tail` antes de reintentar la inserción.

Método enq()

```
public void enq(T value) {  
    Node node = new Node(value);  
    while (true) {  
        Node last = tail.get();  
        Node next = last.next.get();  
        if (last == tail.get()) {  
            if (next == null) {  
                if (last.next.CAS(next, node)) {  
                    tail.CAS(last, node);  
                    return;  
                }  
            } else {  
                tail.CAS(last, next);  
            }  
        }  
    }  
}
```

- Verificar que el último nodo no tenga sucesor (`next == null`).
- Intenta enlazar al nuevo nodo mediante un CAS. Si tiene éxito, la inserción es efectiva.
- Intenta adelantar `tail`. Si falla, significa que otra operación lo ayudó.
- **Ayuda:** Si se detecta que el nodo final tiene un sucesor, intenta adelantar el puntero `tail` antes de reintentar la inserción.

Método enq()

```
public void enq(T value) {
    Node node = new Node(value);
    while (true) {
        Node last = tail.get();
        Node next = last.next.get();
        if (last == tail.get()) {
            if (next == null) {
                if (last.next.CAS(next, node)) {
                    tail.CAS(last, node);
                    return;
                }
            } else {
                tail.CAS(last, next);
            }
        }
    }
}
```

- Verificar que el último nodo no tenga sucesor (`next == null`).
- Intenta enlazar al nuevo nodo mediante un CAS. Si tiene éxito, la inserción es efectiva.
- Intenta adelantar `tail`. Si falla, significa que otra operación lo ayudó.
- **Ayuda:** Si se detecta que el nodo final tiene un sucesor, intenta adelantar el puntero `tail` antes de reintentar la inserción.

Método enq()

```
public void enq(T value) {  
    Node node = new Node(value);  
    while (true) {  
        Node last = tail.get();  
        Node next = last.next.get();  
        if (last == tail.get()) {  
            if (next == null) {  
                if (last.next.CAS(next, node)) {  
                    tail.CAS(last, node);  
                    return;  
                }  
            } else {  
                tail.CAS(last, next);  
            }  
        }  
    }  
}
```

- Verificar que el último nodo no tenga sucesor (`next == null`).
- Intenta enlazar al nuevo nodo mediante un CAS. Si tiene éxito, la inserción es efectiva.
- Intenta adelantar `tail`. Si falla, significa que otra operación lo ayudó.
- **Ayuda:** Si se detecta que el nodo final tiene un sucesor, intenta adelantar el puntero `tail` antes de reintentar la inserción.

Funcionamiento Detallado de deq()

```
public T deq() throws EmptyException {
    while (true) {
        Node first = head.get();
        Node last = tail.get();
        Node next = first.next.get();
        if (first == head.get()) {
            if (first == last) {
                if (next == null) {
                    throw new EmptyException();
                }
                tail.CAS(last, next);
            } else {
                T value = next.value;
                if (head.CAS(first, next))
                    return value;
            }
        }
    }
}
```

- Chequea que no haya interferencias de deq.
- Si la cabeza y la cola coinciden pero next no es nulo, se ayuda adelantando tail.
- Comprueba si es vacía
- Si es vacía, lanza una excepción EmptyException.
- Si la cola tiene elementos, lee el valor del sucesor e intenta avanzar head mediante un CAS.

Funcionamiento Detallado de deq()

```
public T deq() throws EmptyException {
    while (true) {
        Node first = head.get();
        Node last = tail.get();
        Node next = first.next.get();

        if (first == head.get()) {
            if (first == last) {
                if (next == null) {
                    throw new EmptyException();
                }
                tail.CAS(last, next);
            } else {
                T value = next.value;
                if (head.CAS(first, next))
                    return value;
            }
        }
    }
}
```

- Chequea que no haya interferencias de deq concurrentes volviendo a validar la cabeza.
- Si la cabeza y la cola coinciden pero next no es nulo, se ayuda adelantando tail.
- Comprueba de forma segura si la estructura está vacía.
- Si está efectivamente vacía, lanza una excepción EmptyException.
- Si la cola tiene elementos, lee el valor del sucesor e intenta avanzar head mediante un CAS.

Funcionamiento Detallado de deq()

```
public T deq() throws EmptyException {  
    while (true) {  
        Node first = head.get();  
        Node last = tail.get();  
        Node next = first.next.get();  
        if (first == head.get()) {  
            if (first == last) {  
                if (next == null) {  
                    throw new EmptyException();  
                }  
                tail.CAS(last, next);  
            } else {  
                T value = next.value;  
                if (head.CAS(first, next))  
                    return value;  
            }  
        }  
    }  
}
```

- Chequea que no haya interferencias de deq concurrentes volviendo a validar la cabeza.
- Si la cabeza y la cola coinciden pero next no es nulo, se ayuda adelantando tail.
- Comprueba de forma segura si la estructura está vacía.
- Si está efectivamente vacía, lanza una excepción EmptyException.
- Si la cola tiene elementos, lee el valor del sucesor e intenta avanzar head mediante un CAS.

Funcionamiento Detallado de deq()

```
public T deq() throws EmptyException {
    while (true) {
        Node first = head.get();
        Node last = tail.get();
        Node next = first.next.get();
        if (first == head.get()) {
            if (first == last) {
                if (next == null) {
                    throw new EmptyException();
                }
                tail.CAS(last, next);
            } else {
                T value = next.value;
                if (head.CAS(first, next))
                    return value;
            }
        }
    }
}
```

- Chequea que no haya interferencias de deq concurrentes volviendo a validar la cabeza.
- Si la cabeza y la cola coinciden pero next no es nulo, se ayuda adelantando tail.
- Comprueba de forma segura si la estructura está vacía.
- Si está efectivamente vacía, lanza una excepción EmptyException.
- Si la cola tiene elementos, lee el valor del sucesor e intenta avanzar head mediante un CAS.

Funcionamiento Detallado de deq()

```
public T deq() throws EmptyException {
    while (true) {
        Node first = head.get();
        Node last = tail.get();
        Node next = first.next.get();
        if (first == head.get()) {
            if (first == last) {
                if (next == null) {
                    throw new EmptyException();
                }
                tail.CAS(last, next);
            } else {
                T value = next.value;
                if (head.CAS(first, next))
                    return value;
            }
        }
    }
}
```

- Chequea que no haya interferencias de deq concurrentes volviendo a validar la cabeza.
- Si la cabeza y la cola coinciden pero next no es nulo, se ayuda adelantando tail.
- Comprueba de forma segura si la estructura está vacía.
- Si está efectivamente vacía, lanza una excepción `EmptyException`.
- Si la cola tiene elementos, lee el valor del sucesor e intenta avanzar head mediante un CAS.

Funcionamiento Detallado de deq()

```
public T deq() throws EmptyException {  
    while (true) {  
        Node first = head.get();  
        Node last = tail.get();  
        Node next = first.next.get();  
        if (first == head.get()) {  
            if (first == last) {  
                if (next == null) {  
                    throw new EmptyException();  
                }  
                tail.CAS(last, next);  
            } else {  
                T value = next.value;  
                if (head.CAS(first, next))  
                    return value;  
            }  
        }  
    }  
}
```

- Chequea que no haya interferencias de deq concurrentes volviendo a validar la cabeza.
- Si la cabeza y la cola coinciden pero next no es nulo, se ayuda adelantando tail.
- Comprueba de forma segura si la estructura está vacía.
- Si está efectivamente vacía, lanza una excepción EmptyException.
- Si la cola tiene elementos, lee el valor del sucesor e intenta avanzar head mediante un CAS.

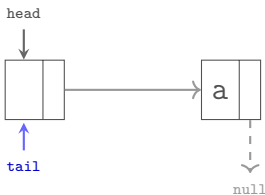
Funcionamiento Detallado de deq()

```
public T deq() throws EmptyException {
    while (true) {
        Node first = head.get();
        Node last = tail.get();
        Node next = first.next.get();
        if (first == head.get()) {
            if (first == last) {
                if (next == null) {
                    throw new EmptyException();
                }
                tail.CAS(last, next);
            } else {
                T value = next.value;
                if (head.CAS(first, next))
                    return value;
            }
        }
    }
}
```

- Chequea que no haya interferencias de deq concurrentes volviendo a validar la cabeza.
- Si la cabeza y la cola coinciden pero next no es nulo, se ayuda adelantando tail.
- Comprueba de forma segura si la estructura está vacía.
- Si está efectivamente vacía, lanza una excepción EmptyException.
- Si la cola tiene elementos, lee el valor del sucesor e intenta avanzar head mediante un CAS.

Técnica de Ayuda: tail retrasado en `deq()`

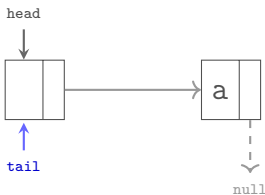
Condición: `first == last` y `next != null` | Estado: Inconsistencia inicial



- **El problema:** Un productor enlazó el nodo **a** pero se suspendió antes de actualizar `tail`. El puntero `tail` quedó rezagado (*lagging behind*).

Técnica de Ayuda: tail retrasado en `deq()`

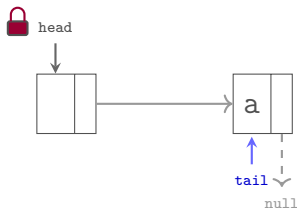
Condición: `first == last` y `next != null` | Estado: `deq()` detecta retraso



- **Detección:** Un consumidor llama a `deq()` y nota que `first == last` (`head == tail`) pero su sucesor `next` no es nulo. Sabe que no puede remover el centinela todavía.

Técnica de Ayuda: tail retrasado en deq()

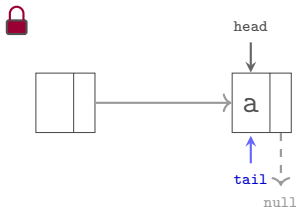
Condición: `first == last` y `next != null` | Estado: `deq()` ayuda y avanza
tail



- **Técnica de Ayuda:** El consumidor ejecuta `tail.CAS(last, next)` para acomodar la estructura del productor flojo, moviendo `tail` de forma segura hacia el nodo **a**.

Técnica de Ayuda: tail retrasado en `deq()`

Condición: `first == last` y `next != null` | Estado: `deq()` linealiza y avanza
head



- **Linealización de `deq`:** Con la cola ya consistente, el consumidor ejecuta con éxito su propio `head.CAS(first, next)`, transformando al nodo **a** en el nuevo centinela y retornando su valor.

Inserción: `enq()`

- `enq()` se linealiza cuando se modifica con éxito el puntero del último nodo (CAS exitoso sobre `last.next`)

Eliminación: `deq()`

- **exitosa (Retorna valor):** Se linealiza cuando se modifica con éxito el puntero de la cabeza (`head.CAS(first, next)`).
- **no exitoso (`EmptyException`):** Se linealiza cuando se lee `next` del centinela.

Cola es Lock-Free

- Cada método verifica primero si existe una operación `enq()` incompleta e intenta terminarla.
- En el peor de los casos, todos los procesos intentan avanzar el campo `tail` de la cola, y al menos uno de ellos **debe tener éxito**.
- Un hilo falla al intentar encolar o desencolar un nodo **solo si** la operación de otro hilo tiene éxito al modificar la referencia. Por lo tanto, siempre hay algún método que logra completar su trabajo.

Motivaciones

- **Limitaciones del lenguaje:** Lenguajes como C o C++ no tienen garbage collector.
- **Eficiencia:** Puede ser más eficiente que una clase gestione sus propios nodos cuando se crean y liberan constantemente múltiples objetos pequeños.
- **Garantías de progreso:** Si el garbage collector no es *lock-free*.

Dinámica de la freeList Local

Reciclaje

Cada hilo administra su propia lista privada y local de nodos no usados.

```
ThreadLocal<Node> freeList = new ThreadLocal<Node>() {  
    protected Node initialValue() { return null; };  
};
```

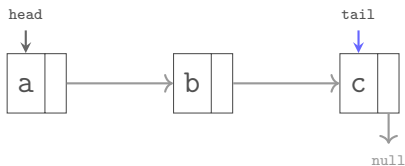
Ciclo de Vida de los Nodos

- **Asignación en enq():** Cuando un productor necesita un nodo, intenta extraerlo de su lista local. Si la lista está vacía, crea uno nuevo (`new`).
- **Devolución en deq():** Cuando un consumidor desencola y descarta un nodo, lo agrega a la lista local para su reutilización futura.

Limitaciones

- Funciona bien bajo una cantidad balanceada de inserciones y extracciones.
- Otras situaciones requieren técnicas como de robo de nodos (*node stealing*).

El Problema ABA con Reciclaje Manual

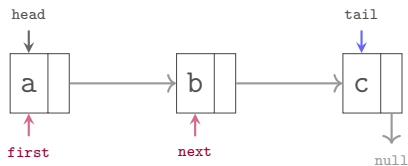


Pool



- **Estado Inicial:** La estructura cuenta con el centinela **a** y los elementos **b** y **c** en la cola.

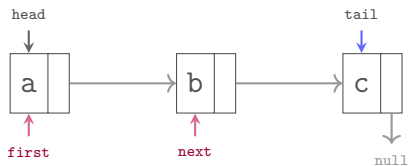
El Problema ABA con Reciclaje Manual



Pool

- **Paso 1:** El Thread 1 inicia un `deq()`, lee los punteros locales `first = a` y `next = b`, y es instantáneamente **suspendido** antes de ejecutar su CAS.

El Problema ABA con Reciclaje Manual

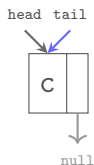


Pool

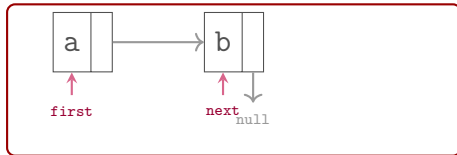


- **Interferencia del Thread 2 (Paso A):** Un segundo hilo concurrente interviene y ejecuta de forma veloz **dos operaciones** `deq()` consecutivas.

El Problema ABA con Reciclaje Manual

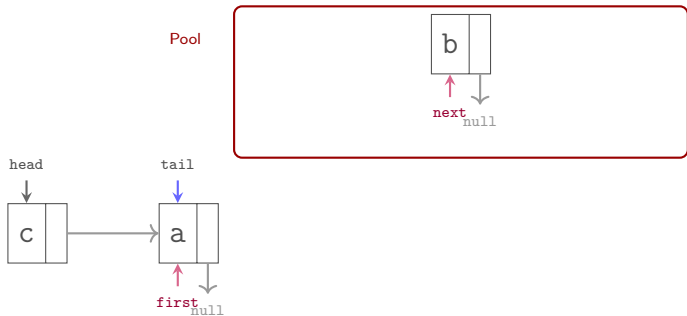


Pool



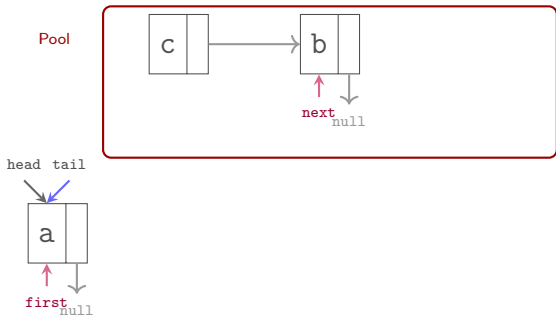
- **Efecto del Desencolado:** Los nodos **a** y **b** son removidos físicamente de la cola abstracta y se alojan en el Pool de reciclaje. head y tail ahora apuntan a **c**.

El Problema ABA con Reciclaje Manual



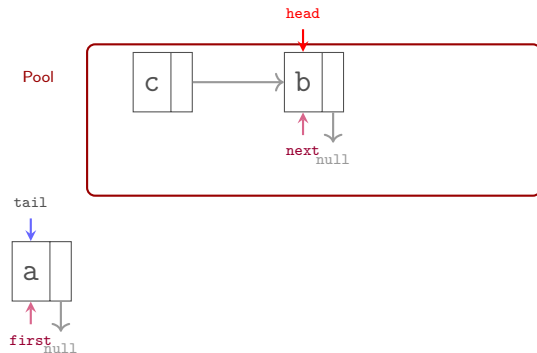
- **Reinserción:** El Thread 2 realiza un `enq()`. Extrae el nodo **a** del pool y lo conecta detrás de **c**. La cola vuelve a tener la dirección de memoria **A** al final.

El Problema ABA con Reciclaje Manual



- Otro hilo ejecuta un `deq()` adicional. El nodo **c** es removido y va al Pool.

El Problema ABA con Reciclaje Manual



- El Thread 1 despierta y ejecuta su CAS postergado. Como head es first, el **CAS tiene éxito** y desplaza la cabeza hacia **b** (next), apuntando a un elemento en el Pool.

Solución al Problema ABA: AtomicStampedReference

Sello (Versiones)

A cada referencia atómica se le pone un sello (*stamp*) que actúa como número de versión.

- Clase AtomicStampedReference<T> que encapsula una referencia al objeto de tipo T y un entero que representa el sello
- **Control de Versiones:** Cada vez que se modifica la referencia, el valor del sello se incrementa. Aunque una dirección
- Aunque se vuelva a la misma referencia, los sellos son distintos y el CAS fallará.

API de AtomicStampedReference

- Provee los siguientes métodos atómicos.

```
public boolean compareAndSet(  
    T expectedReference,  
    T newReference,  
    int expectedStamp,  
    int newStamp  
);  
  
public T get(int[] stampHolder);
```

```
public T getReference();  
  
public int getStamp();  
  
public void set(  
    T newReference,  
    int newStamp  
);
```

Método `deq()` empleando Sellos de Versión

```
public T deq() throws EmptyException {
    int[] lastStamp = new int[1];
    int[] firstStamp = new int[1];
    int[] nextStamp = new int[1];
    while (true) {
        Node first = head.get(firstStamp);
        Node last = tail.get(lastStamp);
        Node next = first.next.get(nextStamp);
        if (head.getStamp() == firstStamp[0]) {
            if (first == last) {
                if (next == null) {
                    throw new EmptyException();
                }
                tail.compareAndSet(last, next,
                    lastStamp[0], lastStamp[0]+1);
            } else {
                T value = next.value;
                if (head.compareAndSet(first, next,
                    firstStamp[0], firstStamp[0]+1)) {
                    free(first);
                    return value;
                }
            }
        }
    }
}
```

- Se leen las referencias y los sellos de versión.

Método `deq()` empleando Sellos de Versión

```
public T deq() throws EmptyException {
    int[] lastStamp = new int[1];
    int[] firstStamp = new int[1];
    int[] nextStamp = new int[1];
    while (true) {
        Node first = head.get(firstStamp);
        Node last = tail.get(lastStamp);
        Node next = first.next.get(nextStamp);

        if (head.getStamp() == firstStamp[0]) {
            if (first == last) {
                if (next == null) {
                    throw new EmptyException();
                }
                tail.compareAndSet(last, next,
                    lastStamp[0], lastStamp[0]+1);
            } else {
                T value = next.value;
                if (head.compareAndSet(first, next,
                    firstStamp[0], firstStamp[0]+1)) {
                    free(first);
                    return value;
                }
            }
        }
    }
}
```

- Se verifica que el sello de la cabeza no haya cambiado.

Método `deq()` empleando Sellos de Versión

```
public T deq() throws EmptyException {
    int[] lastStamp = new int[1];
    int[] firstStamp = new int[1];
    int[] nextStamp = new int[1];
    while (true) {
        Node first = head.get(firstStamp);
        Node last = tail.get(lastStamp);
        Node next = first.next.get(nextStamp);
        if (head.getStamp() == firstStamp[0]) {
            if (first == last) {
                if (next == null) {
                    throw new EmptyException();
                }
                tail.compareAndSet(last, next,
                    lastStamp[0], lastStamp[0]+1);
            }
        }
        else {
            T value = next.value;
            if (head.compareAndSet(first, next,
                firstStamp[0], firstStamp[0]+1)) {
                free(first);
                return value;
            }
        }
    }
}
```

- Al avanzar los punteros (`tail` o `head`), se exige que el sello no haya cambiado y se incrementa en `+1`.

Método enq() empleando Sellos de Versión

```
public void enq(T value) {
    Node node = new Node(value);
    int[] lastStamp = new int;
    int[] nextStamp = new int;
    while (true) {
        Node last = tail.get(lastStamp);
        Node next = last.next.get(nextStamp);

        if (last == tail.get()) {
            if (next == null) {
                if (last.next.CAS(next, node,
                                nextStamp, nextStamp+1)) {
                    tail.CAS(last, node,
                            lastStamp, lastStamp+1);
                    return;
                }
            } else {
                tail.CAS(last, next,
                        lastStamp, lastStamp+1);
            }
        }
    }
}
```

- **Lectura de Sellos:** Al igual que en la remoción, el productor captura simultáneamente los nodos last y next junto con sus sellos de versión vigentes.

Método enq() empleando Sellos de Versión

```
public void enq(T value) {
    Node node = new Node(value);
    int[] lastStamp = new int;
    int[] nextStamp = new int;
    while (true) {
        Node last = tail.get(lastStamp);
        Node next = last.next.get(nextStamp);
        if (last == tail.get()) {
            if (next == null) {
```

```
                if (last.next.CAS(next, node,
                                    nextStamp, nextStamp+1)) {
                    tail.CAS(last, node,
                             lastStamp, lastStamp+1);
                    return;
                }
            }
        }
    }
```

- **Enlace Físico con Incremento:** El primer CAS exige que el sello de next coincida con el leído (que debería ser null). Al enlazar el nodo, incrementa su versión para alertar a otros hilos.

Método enq() empleando Sellos de Versión

```
public void enq(T value) {
    Node node = new Node(value);
    int[] lastStamp = new int;
    int[] nextStamp = new int;
    while (true) {
        Node last = tail.get(lastStamp);
        Node next = last.next.get(nextStamp);
        if (last == tail.get()) {
            if (next == null) {
```

```
                if (last.next.CAS(next, node,
                                    nextStamp, nextStamp+1)) {
                    tail.CAS(last, node,
                             lastStamp, lastStamp+1);
                    return;
                }
            }
        }
    }
}
```

- El productor (o un ayudante) avanza el puntero tail validando su sello original e incrementándolo en +1.

Cola síncrona (capacidad 0)

Rendezvous

- Uno o más productores generan elementos para ser removidos, en orden FIFO, por uno o más hilos consumidores.
- Los productores y los consumidores realizan un *rendezvous*: un productor que coloca un elemento en la cola se bloquea hasta que dicho elemento sea removido por un consumidor, y viceversa.

Cola Síncrona (Monitor): Representación

```
public class SynchronousQueue<T> {  
    T item = null;  
    boolean enqueueing;  
    Lock lock;  
    Condition condition;  
}
```

Wake-ups espúreos

- Al usar una única condición, cada notificación despierta a productores y a consumidores.
- Se puede mitigar usando varias condiciones, pero se debe bloquear toda la estructura.

Cola Síncrona (Monitor): Método enq()

```
public void enq(T value) {  
    lock.lock();  
    try {  
        while (enqueueing)  
            condition.await();  
        enqueueing = true;  
        item = value;  
        condition.signalAll();  
        while (item != null)  
            condition.await();  
        enqueueing = false;  
        condition.signalAll();  
    } finally {  
        lock.unlock();  
    }  
}
```

- Si hay otro productor esperando, se bloquea en la condición.
- Tras depositar el elemento y despertar a los demás hilos, se bloquea hasta que un consumidor retire el objeto.
- Cuando el encuentro se concreta, el productor libera la bandera enqueueing y despierta a los productores en espera.

Cola Síncrona (Monitor): Método enq()

```
public void enq(T value) {  
    lock.lock();  
    try {  
        while (enqueueing)  
            condition.await();  
        enqueueing = true;  
        item = value;  
        condition.signalAll();  
  
        while (item != null)  
            condition.await();  
  
        enqueueing = false;  
        condition.signalAll();  
    } finally {  
        lock.unlock();  
    }  
}
```

- Si hay otro productor esperando, se bloquea en la condición.
- Tras depositar el elemento y despertar a los demás hilos, se bloquea hasta que un consumidor retire el objeto.
- Cuando el encuentro se concreta, el productor libera la bandera enqueueing y despierta a los productores en espera.

Cola Síncrona (Monitor): Método enq()

```
public void enq(T value) {  
    lock.lock();  
    try {  
        while (enqueueing)  
            condition.await();  
        enqueueing = true;  
        item = value;  
        condition.signalAll();  
        while (item != null)  
            condition.await();  
        enqueueing = false;  
        condition.signalAll();  
    } finally {  
        lock.unlock();  
    }  
}
```

- Si hay otro productor esperando, se bloquea en la condición.
- Tras depositar el elemento y despertar a los demás hilos, se bloquea hasta que un consumidor retire el objeto.
- Cuando el encuentro se concreta, el productor libera la bandera enqueueing y despierta a los productores en espera.

Cola Síncrona (Monitor): Método deq()

```
public T deq() {  
    lock.lock();  
    try {  
        while (item == null)  
            condition.await();  
        T t = item;  
        item = null;  
        condition.signalAll();  
        return t;  
    } finally {  
        lock.unlock();  
    }  
}
```

- Adquiere la cerradura y, si no hay ningún elemento disponible, se bloquea esperando a un productor.
- Remueve la referencia del objeto, vacía el campo fijándolo en null y despierta al productor para confirmarle que su elemento ya fue tomado.
- Retorna el valor obtenido.

Cola Síncrona (Monitor): Método deq()

```
public T deq() {  
    lock.lock();  
    try {  
        while (item == null)  
            condition.await();  
        T t = item;  
        item = null;  
        condition.signalAll();  
        return t;  
    } finally {  
        lock.unlock();  
    }  
}
```

- Adquiere la cerradura y, si no hay ningún elemento disponible, se bloquea esperando a un productor.
- Remueve la referencia del objeto, vacía el campo fijándolo en `null` y despierta al productor para confirmarle que su elemento ya fue tomado.
- Retorna el valor obtenido.

Cola Síncrona (Monitor): Método `deq()`

```
public T deq() {  
    lock.lock();  
    try {  
        while (item == null)  
            condition.await();  
        T t = item;  
        item = null;  
        condition.signalAll();  
        return t;  
    } finally {  
        lock.unlock();  
    }  
}
```

- Adquiere la cerradura y, si no hay ningún elemento disponible, se bloquea esperando a un productor.
- Remueve la referencia del objeto, vacía el campo fijándolo en `null` y despierta al productor para confirmarle que su elemento ya fue tomado.
- Retorna el valor obtenido.

Estructuras de Datos Duales (DualStructures)

Concepto General

Trata `enq()` y `deq()` simétricamente. Si un consumidor encuentra la cola vacía, divide la operación en dos pasos:

- **Reservación:** Inserta un objeto de reservación en la cola indicando que espera un productor.
- **Espera Pasiva:** El consumidor hace *spinning* local sobre un espacio vacío hasta que un productor deposita el elemento.

Exclusión Mutua de Tipos

La cola solo puede contener elementos en espera o reservaciones en espera, nunca ambos tipos en simultáneo.

Estructura Dual: Método enq()

```
public void enq(T e) {
    Node offer = new Node(e, NodeType.ITEM);
    while (true) {
        Node t = tail.get(), h = head.get();
        if (h == t || t.type == NodeType.ITEM) {
            Node n = t.next.get();
            if (t == tail.get()) {
                if (n != null) {
                    tail.CAS(t, n);
                } else if (t.next.CAS(n, offer)) {
                    tail.CAS(t, offer);
                }
            }
            while (offer.item.get() == e);
            h = head.get();
            if (offer == h.next.get())
                head.CAS(h, offer);
            return;
        }
    }
}
```

```
    } else {
        Node n = h.next.get();
        if (t != tail.get() ||
            h != head.get() || n == null) {
            continue;
        }
        boolean success =
            n.item.CAS(null, e);
        head.CAS(h, n);
        if (success) return;
    }
}
```

- Si la cola está vacía o contiene otros elementos, el productor enlaza su nodo mediante un CAS, avanza la cola y hace *spinning* local hasta que un consumidor retire el dato.

Estructura Dual: Método enq()

```
public void enq(T e) {
    Node offer = new Node(e, NodeType.ITEM);
    while (true) {
        Node t = tail.get(), h = head.get();
        if (h == t || t.type == NodeType.ITEM) {
            Node n = t.next.get();
            if (t == tail.get()) {
                if (n != null) {
                    tail.CAS(t, n);
                } else if (t.next.CAS(n, offer)) {
                    tail.CAS(t, offer);
                    while (offer.item.get() == e);
                    h = head.get();
                    if (offer == h.next.get())
                        head.CAS(h, offer);
                    return;
                }
            }
        }
    }
}
```

```
    } else {
        Node n = h.next.get();
        if (t != tail.get() ||
            h != head.get() || n == null) {
            continue;
        }
        boolean success =
            n.item.CAS(null, e);
        head.CAS(h, n);
        if (success) return;
    }
}
```

- **Consumación de Reservación:** Si la cola contiene reservaciones, el productor toma la primera de la lista e intenta depositar el elemento e

Estructura Dual: Método enq()

```
public void enq(T e) {
    Node offer = new Node(e, NodeType.ITEM);
    while (true) {
        Node t = tail.get(), h = head.get();
        if (h == t || t.type == NodeType.ITEM) {
            Node n = t.next.get();
            if (t == tail.get()) {
                if (n != null) {
                    tail.CAS(t, n);
                } else if (t.next.CAS(n, offer)) {
                    tail.CAS(t, offer);
                    while (offer.item.get() == e);
                    h = head.get();
                    if (offer == h.next.get())
                        head.CAS(h, offer);
                    return;
                }
            }
        }
    }
}
```

```
    } else {
        Node n = h.next.get();
        if (t != tail.get() ||
            h != head.get() || n == null) {
            continue;
        }
        boolean success =
            n.item.CAS(null, e);
        head.CAS(h, n);
        if (success) return;
    }
}
```

- **Avanzar Cabeza:** Sin importar si el CAS del dato tuvo éxito o fue ganado por otro productor concurrente, el hilo actual colabora avanzando el puntero

Estructuras LIFO: Introducción a las Pilas Concurrentes

Definición de la Clase Stack<T>

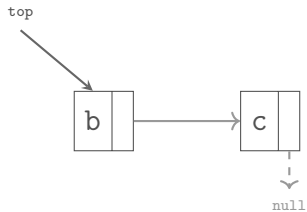
Una pila es una colección de elementos (de tipo T) que expone los métodos `push()` y `pop()` bajo la propiedad estricta **LIFO** (*Last-In-First-Out*): el último elemento en ingresar es el primero en ser removido.

El Desafío de la Concurrencia

A primera vista, las pilas parecen ofrecer poca posibilidad de optimización concurrente debido a su estructura:

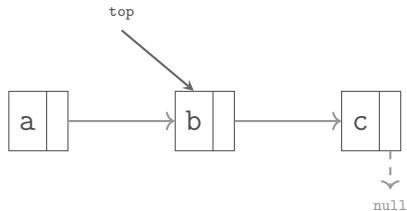
- Las inserciones y las extracciones necesitan sincronizar en un único punto crítico: **el tope de la pila**
- Esta centralización genera un alto nivel de contención en escenarios de alta concurrencia.

Pila (libre de lock): Operación push(a)



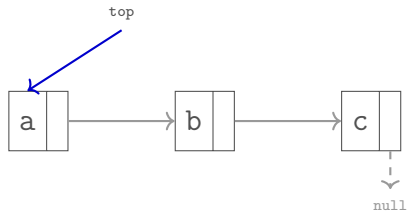
- Estado inicial

Pila (libre de lock): Operación push(a)



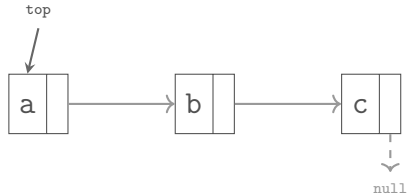
- Se asigna memoria para el nuevo nodo con el valor **a** y se fija el link a top. No se cambi top.

Pila (libre de lock): Operación push(a)



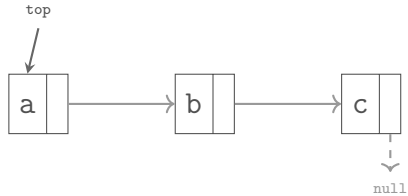
- Se ejecuta un CAS para modificar a top. Al tener éxito, la referencia global se desplaza hacia **a**, consumando la inserción.

Pila (libre de lock): Operación pop()



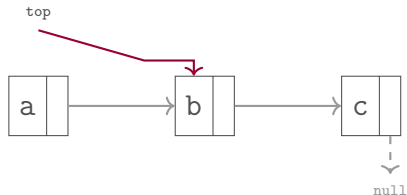
- Al invocar `pop()` se lee el tope actual de la pila.

Pila (libre de lock): Operación pop()



- Se almacena el valor **a** del nodo y next.

Pila (libre de lock): Operación pop()



- Intenta adelantar la referencia global ejecutando CAS. Si tiene éxito, el nodo se elimina de la pila y se retorna su valor.

Estructura de la Pila Libre de Bloqueos

Representación de Componentes

La clase delega el estado del tope a una referencia atómica e incorpora una estrategia de *backoff* para mitigar la contención en el hardware.

```
public class LockFreeStack<T> {  
    AtomicReference<Node> top =  
        new AtomicReference<Node>(null);  
  
    static final int MIN_DELAY = ...;  
    static final int MAX_DELAY = ...;  
  
    Backoff backoff =  
        new Backoff(MIN_DELAY, MAX_DELAY);  
}
```

```
public class Node {  
    public T value;  
    public Node next;  
  
    public Node(T value) {  
        this.value = value;  
        this.next = null;  
    }  
}
```

Método push() con Estrategia de Backoff

```
protected boolean tryPush(Node node) {  
    Node oldTop = top.get();  
    node.next = oldTop;  
    return top.compareAndSet(oldTop,node);  
}
```

```
public void push(T value) {  
    Node node = new Node(value);  
    while (true) {  
        if (tryPush(node)) {  
            return;  
        } else {  
            backoff.backoff();  
        }  
    }  
}
```

- **Punto de Linealización:** Ocurre en el instante en que el método `compareAndSet` dentro de `tryPush()` redirige exitosamente la referencia `top` hacia el nuevo nodo.
- **Manejo de Contención:** Si el intento de inserción física en `tryPush()` falla por colisión concurrente, el hilo se suspende temporalmente mediante el `backoff`.

Método push() con Estrategia de Backoff

```
protected boolean tryPush(Node node) {  
    Node oldTop = top.get();  
    node.next = oldTop;  
    return top.compareAndSet(oldTop,node);  
}
```

```
public void push(T value) {  
    Node node = new Node(value);  
    while (true) {  
        if (tryPush(node)) {  
            return;  
        } else {  
            backoff.backoff();  
        }  
    }  
}
```

- **Punto de Linealización:** Ocurre en el instante en que el método `compareAndSet` dentro de `tryPush()` redirige exitosamente la referencia `top` hacia el nuevo nodo.
- **Manejo de Contención:** Si el intento de inserción física en `tryPush()` falla por colisión concurrente, el hilo se suspende temporalmente mediante el `backoff`.

Método pop() con Estrategia de Backoff

```
protected Node tryPop() throws EmptyException {
    Node oldTop = top.get();
    if (oldTop == null) {
        throw new EmptyException();
    }
    Node newTop = oldTop.next;
    if (top.compareAndSet(oldTop, newTop)) {
        return oldTop;
    } else {
        return null;
    }
}

public T pop() throws EmptyException {
    while (true) {
        Node returnNode = tryPop();
        if (returnNode != null) {
            return returnNode.value;
        } else {
            backoff.backoff();
        }
    }
}
```

- **Punto de Linealización:** Ocurre en el instante en que el método `compareAndSet` dentro de `tryPop()` redirige exitosamente la referencia `top` hacia el nodo sucesor.
- **Manejo de Contención:** Si el intento de extracción falla por conflicto de concurrencia y retorna `null`, el método `pop()` suspende temporalmente al hilo en el backoff.

Método pop() con Estrategia de Backoff

```
protected Node tryPop() throws EmptyException {
    Node oldTop = top.get();
    if (oldTop == null) {
        throw new EmptyException();
    }
    Node newTop = oldTop.next;
    if (top.compareAndSet(oldTop, newTop)) {
        return oldTop;
    } else {
        return null;
    }
}

public T pop() throws EmptyException {
    while (true) {
        Node returnNode = tryPop();
        if (returnNode != null) {
            return returnNode.value;
        } else {
            backoff.backoff();
        }
    }
}
```

- **Punto de Linealización:** Ocurre en el instante en que el método `compareAndSet` dentro de `tryPop()` redirige exitosamente la referencia `top` hacia el nodo sucesor.
- **Manejo de Contención:** Si el intento de extracción falla por conflicto de concurrencia y retorna `null`, el método `pop()` suspende temporalmente al hilo en el backoff.

Estrategia de Espera Exponencial (Backoff)

Principio del backoff

La estrategia de *backoff* introduce un retardo antes de permitir que el hilo fallido vuelva a reintentar la operación.

Dinámica Exponencial

- **Retraso Inicial:** El hilo se suspende por un período breve configurado por `MIN_DELAY`.
- **Crecimiento:** Con cada colisión consecutiva detectada por el CAS, el rango de espera se duplica exponencialmente.
- **Límite:** El tiempo máximo de suspensión queda acotado estrictamente por el valor `MAX_DELAY`.

Propiedad Lock-Free

La pila es *lock-free*: Un hilo falla al completar un método `push()` o `pop()` **únicamente** si existen infinitas modificaciones exitosas concurrentes de `top`.

Puntos de Linearización

- `push` y `pop` exitoso: cuando se ejecuta exitosamente el `compareAndSet`.
- `top` no exitoso: cuando se observa `null` al leer `pop()` en una pila vacía.

El Principio de Eliminación

El Cuello de Botella Secuencial

El *backoff* no resuelve la secuencialidad.

El Principio de Eliminación

El Cuello de Botella Secuencial

El *backoff* no resuelve la secuencialidad.

Principio de eliminación

Un `push()` seguido inmediatamente por un `pop()` se cancelan mutuamente: el estado de la pila no cambia.

- un productor puede entregarle su valor directamente a un consumidor concurrente sin tocar la pila.
- Las operaciones se **eliminan** entre sí.
- **Linearización:** El sistema combinado es linearizable.
 - Las llamadas eliminadas se ordenan en el instante exacto en que intercambian sus valores.

Eliminación usando un arreglo

El EliminationArray

- Estructura auxiliar que se usa para que productor y consumidor intercambien valores.
- Los hilos eligen posiciones aleatorias en un arreglo para intercambiar.
- Si la operación no logra eliminarse (no halla socio o encuentra el rol equivocado, como un `push()` con otro `push()`), intenta en otra posición o accede a la pila central.

Elimination back-off stack

Idea

- Usa un `EliminationArray` como un esquema de back-off
 - Cada hilo accede primero a la `LockFreeStack` y, si no logra completar su llamada , intenta eliminar utilizando el arreglo.
 - Si no consigue eliminarse, vuelve a invocar a la `LockFreeStack`, y así sucesivamente.

Intercambiador Libre de Bloqueos (LockFreeExchanger)

Principio de Operación

Permite a dos hilos intercambiar valores de tipo T. El primer hilo en llegar deposita su valor y hace *spinning* con un tiempo límite (*timeout*). El segundo hilo toma el elemento, deposita el suyo y confirma el encuentro.

Intercambiador Libre de Bloqueos (LockFreeExchanger)

Principio de Operación

Permite a dos hilos intercambiar valores de tipo T. El primer hilo en llegar deposita su valor y hace *spinning* con un tiempo límite (*timeout*). El segundo hilo toma el elemento, deposita el suyo y confirma el encuentro.

Máquina de Estados Basada en Sellos

- **EMPTY (Vacío):** No hay ningún hilo esperando. El canal está libre para iniciar un encuentro.
- **WAITING (Esperando):** Un hilo llegó primero, depositó su objeto y se encuentra esperando un socio.
- **BUSY (Ocupado):** Dos hilos están activamente consolidando un intercambio en este espacio.

Estructura de LockFreeExchanger

Representación

Una referencia de atómica con sello.

```
public class LockFreeExchanger<T> {  
    static final int EMPTY = ..., WAITING = ..., BUSY = ...;  
  
    AtomicStampedReference<T> slot =  
        new AtomicStampedReference<T>(null, 0);  
}
```

Variables Globales

- EMPTY, WAITING, BUSY: Constantes que representan los estados.
- slot: Referencia de tipo AtomicStampedReference que resguarda el elemento actual y cuya estampa registra el estado interno.

Funcionamiento de LockFreeExchanger (1/2)

Caso EMPTY:

```
public T exchange(T myItem) {  
    int[] stampHolder = {EMPTY};  
    while (true) {  
        T yrItem = slot.get(stampHolder);  
        int stamp = stampHolder;  
        switch(stamp) {  
            case EMPTY:  
                if (slot.CAS(yrItem, myItem, EMPTY, WAITING)) {  
                    while (true) {  
                        yrItem = slot.get(stampHolder);  
                        if (stampHolder == BUSY) {  
                            slot.set(null, EMPTY);  
                            return yrItem;  
                        }  
                    }  
                }  
                break;  
            case WAITING: ... // Oculto para analisis  
            case BUSY: break;  
        }  
    }  
}
```

- El primer hilo cambia el estado a WAITING y deposita su elemento.
- Queda en un bucle de *spinning* esperando a un socio concurrente.
- Cuando el socio cambia el estado a BUSY, consume el nuevo valor, resetea a EMPTY y retorna.

Funcionamiento de LockFreeExchanger (2/2)

```
public T exchange(T myItem) {  
    int[] stampHolder = {EMPTY};  
    while (true) {  
        T yrItem = slot.get(stampHolder);  
        int stamp = stampHolder;  
        switch(stamp) {  
            case EMPTY: ... // Oculto para analisis  
            case WAITING:  
                if (slot.CAS(yrItem, myItem, WAITING, BUSY))  
                    return yrItem;  
                break;  
            case BUSY: break;  
        }  
    }  
}
```

- Ya existe un hilo esperando en la ranura con su elemento guardado.
- El hilo actual intenta intercambiar los elementos atómicamente transicionando de WAITING a BUSY.
- Si el CAS tiene éxito, se concreta el encuentro y retorna el elemento del rival.

Soporte de Timeout en LockFreeExchanger

Gestión de Tiempo:

```
public T exchange(T myItem, long timeout, TimeUnit
    unit) throws TimeoutException {
    long nanos = unit.toNanos(timeout);

    long timeBound = System.nanoTime() + nanos;

    int[] stampHolder = {EMPTY};
    while (true) {
        if (System.nanoTime() > timeBound)
            throw new TimeoutException();
        // ... (Lectura de slot y switch) ...
        case EMPTY:
            if (slot.CAS(yrItem, myItem, EMPTY, WAITING)) {
                while (System.nanoTime() < timeBound) {
                    // ... (Esperando al socio en BUSY) ...
                }

                if (slot.CAS(myItem, null, WAITING, EMPTY))
                    throw new TimeoutException();
                else { // otro tomo el elemento
                    ...
                }
            }
        }
    }
}
```

- Se calcula el límite de tiempo (timeBound) antes de entrar al ciclo.
- En estado WAITING se espera activamente mientras no se venza el plazo.
- Si expira, intenta retirar su oferta restaurando a EMPTY.
- Si el CAS de retiro falla, significa que otro hilo lo tomó en el último instante; se acepta el intercambio y no se lanza excepción.

Soporte de Timeout en LockFreeExchanger

Gestión de Tiempo:

```
public T exchange(T myItem, long timeout, TimeUnit
    unit) throws TimeoutException {
    long nanos = unit.toNanos(timeout);

    long timeBound = System.nanoTime() + nanos;

    int[] stampHolder = {EMPTY};
    while (true) {
        // ... (Lectura de slot y switch) ...
        case EMPTY:
            if (slot.CAS(yrItem, myItem, EMPTY, WAITING)) {
                // ... (Esperando al socio en BUSY) ...
                if (slot.CAS(myItem, null, WAITING, EMPTY))
                    throw new TimeoutException();
                else { // otro tomo el elemento
                    yrItem = slot.get(stampHolder);
                    slot.set(null, EMPTY);
                    return yrItem;
                }
            }
        }
    }
    break;
}
```

- **Cálculo inicial:** Se calcula el límite de tiempo (timeBound) antes de entrar al ciclo.
- **Spinning acotado:** En estado WAITING se espera activamente mientras no se venza el plazo.
- **Cancelación segura:** Si expira, intenta retirar su oferta restaurando a EMPTY.
- **Carrera crítica:** Si el CAS de retiro falla, significa que un socio lo tomó en el último instante; se acepta el intercambio y no se lanza excepción.

Puntos de Linearización

- **Intercambio Exitoso:** Ocurre cuando el **segundo hilo** cambia con éxito el estado de WAITING a BUSY.
- **Intercambio Fallido:** Se lineariza en el instante en el que el temporizador expira y se lanza la excepción.

Puntos de Linearización

- **Intercambio Exitoso:** Ocurre cuando el **segundo hilo** cambia con éxito el estado de WAITING a BUSY.
- **Intercambio Fallido:** Se lineariza en el instante en el que el temporizador expira y se lanza la excepción.

Progreso

- **Lock-Free:** Si llamadas concurrentes a `exchange()` disponen de suficiente tiempo solo fallarán si otros intercambios están teniendo éxito de forma repetida.
- **Duración del Timeout:** Un tiempo de espera demasiado corto puede provocar que un hilo nunca logre completar un intercambio.

Estructura de EliminationArray

Principio de Diseño

Un arreglo de objetos `LockFreeExchanger` donde cada hilo que intenta realizar una eliminación selecciona una posición al azar e invoca al método de intercambio.

Estructura de EliminationArray

Principio de Diseño

Un arreglo de objetos LockFreeExchanger donde cada hilo que intenta realizar una eliminación selecciona una posición al azar e invoca al método de intercambio.

- **Subrango Dinámico:** Cada hilo selecciona una posición aleatoria dentro de un subrango específico (range). Este rango se ajusta dinámicamente según la carga y nivel de contención del sistema.
- **Generador de Números Aleatorios:** Cada hilo debe usar su propio generador local.
- **Resultado:** El método retorna el elemento provisto por otro hilo, o lanza una excepción cuando expira el tiempo límite.

Código de EliminationArray

```
public class EliminationArray<T> {  
    private static final int duration = ...;  
    LockFreeExchanger<T>[] exchanger;  
  
    public EliminationArray(int capacity) {  
        exchanger = (LockFreeExchanger<T>[])  
            new LockFreeExchanger[capacity];  
        for (int i = 0; i < capacity; i++) {  
            exchanger[i] =  
                new LockFreeExchanger<T>();  
        }  
    }  
  
    public T visit(T value, int range)  
        throws Exception {  
        int slot = ThreadLocalRandom.current().nextInt(range);  
        return exchanger[slot].exchange(value,  
            duration, MILLISECONDS);  
    }  
}
```

- Selección de la posición:
 - Determina la posición en el arreglo.

Código de EliminationArray

```
public class EliminationArray<T> {  
    private static final int duration = ...;  
    LockFreeExchanger<T>[] exchanger;  
  
    public EliminationArray(int capacity) {  
        exchanger = (LockFreeExchanger<T>[])  
            new LockFreeExchanger[capacity];  
        for (int i = 0; i < capacity; i++) {  
            exchanger[i] =  
                new LockFreeExchanger<T>();  
        }  
    }  
  
    public T visit(T value, int range)  
        throws Exception {  
        int slot = ThreadLocalRandom.current()  
            return exchanger[slot].exchange(value,  
                duration, MILLISECONDS);  
    }  
}
```

- **Delegación al Intercambiador:**

- Invoca al elemento LockFreeExchanger ubicado en la celda elegida.
- Le transfiere el valor local y un tiempo de espera fijo predefinido para resolver el encuentro de eliminación.

Pila con Eliminación y Backoff (`EliminationBackoffStack`)

Principio de Operación

Es una subclase de `LockFreeStack` que sobrescribe `push()` y `pop()`. En lugar de usar un retardo cuando falla el acceso a la pila, utiliza un `EliminationArray` para intentar cruzar las operaciones concurrentes.

- **Asimetría:** Al invocar `visit()`, los hilos productores envían el elemento de tipo `T`, mientras que los consumidores envían la referencia `null`.
- **Verificación:**
 - Si un `push()` concreta el intercambio y recibe `null`, finaliza con éxito.
 - Si un `pop()` concreta el intercambio y recibe un valor distinto de `null`, obtiene el dato del productor y finaliza.
- **Política de Rango (`RangePolicy`):** Un objeto local por hilo regula dinámicamente el subrango de colisión. El rango se reduce si ocurren *timeouts* y se expande ante encuentros exitosos.

Método push() con Eliminación

```
public void push(T value) {
    RangePolicy rangePolicy = policy.get();
    Node node = new Node(value);
    while (true) {
        if (tryPush(node)) {
            return;
        } else try {
            T otherValue = eliminationArray
                .visit(value, rangePolicy.getRange());
            if (otherValue == null) {
                rangePolicy.recordSuccess();
                return; // Exchanged with pop
            }
        } catch (TimeoutException ex) {
            rangePolicy.recordTimeout();
        }
    }
}
```

- **Intento Alternativo:**

- Si la inserción directa en el tope con tryPush() falla usa el eliminationArray entregando su valor usando el rango que le dicta su política local.

Método push() con Eliminación

```
public void push(T value) {
    RangePolicy rangePolicy = policy.get();
    Node node = new Node(value);
    while (true) {
        if (tryPush(node)) {
            return;
        } else try {
            T otherValue = eliminationArray
                .visit(value, rangePolicy.getRange());

            if (otherValue == null) {

                rangePolicy.recordSuccess();

                return; // Exchanged with pop
            }
        } catch (TimeoutException ex) {
            rangePolicy.recordTimeout();
        }
    }
}
```

• Intercambio Exitoso:

- Si el intercambiador le devuelve null, el hilo confirma que un consumidor tomó su nodo.
- Registra el éxito en la política de rango y retorna sin haber tocado la pila central.

Método pop() con Eliminación

```
public T pop() throws Exception {
    RangePolicy rangePolicy = policy.get();
    while (true) {
        Node returnNode = tryPop();
        if (returnNode != null) {
            return returnNode.value;
        } else try {
            T otherValue = eliminationArray
                .visit(null, rangePolicy.getRange());
            if (otherValue != null) {
                rangePolicy.recordSuccess();
                return otherValue;
            }
        } catch (TimeoutException ex) {
            rangePolicy.recordTimeout();
        }
    }
}
```

- Si tryPop() falla debido a colisiones concurrentes en el tope de la pila, el consumidor redirige su esfuerzo hacia el arreglo de eliminación.
- Invoca visit() ofreciendo null como carnada para capturar un productor.

Método pop() con Eliminación

```
public T pop() throws Exception {
    RangePolicy rangePolicy = policy.get();
    while (true) {
        Node returnNode = tryPop();
        if (returnNode != null) {
            return returnNode.value;
        } else try {
            T otherValue = eliminationArray
                .visit(null, rangePolicy.getRange());

            if (otherValue != null) {
                rangePolicy.recordSuccess();
                return otherValue;
            }
        } catch (TimeoutException ex) {
            rangePolicy.recordTimeout();
        }
    }
}
```

- Si obtiene una referencia de datos válida (distinta de null), la eliminación mutua se ha concretado.
- Notifica la efectividad de la ranura a su política y retorna directamente el valor extraído.

Puntos de Linealización

- **Acceso a la Pila Central:** Cualquier llamada exitosa a `push()` o `pop()` que complete operando sobre `LockFreeStack` se lineariza en el cuando hace la modificación en la pila.
- **Operaciones Eliminadas:** Cualquier par de llamadas concurrentes que colisionen y se eliminen en el arreglo se linearizan en el momento en que intercambian sus valores.
- **Independencia:** Las operaciones completadas por eliminación no afectan la linearizabilidad de la pila central. Podrían haber ocurrido en cualquier estado de la `LockFreeStack` y, tras suceder, el estado de la pila central permanece idéntico.